Exercices supplémentaires — `pyDatalog`

IA Hybride et Apprentissage Automatique (L6SOIAAU)

Ljudmila PETKOVIC
Licence « Sciences du langage » (L3)
Sorbonne Université, UFR Sociologie et Informatique pour les Sciences Humaines

10 avril 2026

Objectifs

À l'issue de ce TD, vous devez être capables de :
— distinguer un **fait**, une **règle** et une **requête** en `pyDatalog` ;
— écrire des requêtes simples avec des variables ;
— définir des règles déclaratives ;
— utiliser des fonctions logiques simples ;
— comprendre une agrégation élémentaire.

Énoncé

1. Questions de cours

Répondre brièvement.

1. Quelle est la différence entre un **fait** et une **règle** en `pyDatalog` ?
2. À quoi sert `pyDatalog.create_terms(...)` ?
3. Pourquoi les variables logiques commencent-elles généralement par une majuscule ?
4. Que signifie le symbole `<=` lorsqu'on écrit une règle ?

2. Premières requêtes

On considère le code suivant :

```
from pyDatalog import pyDatalog
pyDatalog.clear()
pyDatalog.create_terms('X,Y')
```

1. Écrire une requête qui demande les valeurs de `X` telles que `X == 3`.
2. Écrire une requête qui impose en même temps :
   — `X == 2`
   — `Y == X + 1`
3. Quel résultat attend-on pour `Y` dans la question précédente ?

3. Faits et requêtes

On considère les faits suivants :

```
from pyDatalog import pyDatalog
pyDatalog.clear()
pyDatalog.create_terms('X, suit, aime')

+suit('Alice', 'Python')
+suit('Bob', 'Python')
+suit('Claire', 'Logique')
+aime('Alice', 'IA')
+aime('Bob', 'Reseaux')
+aime('Claire', 'IA')
```

1. Écrire une requête qui donne toutes les personnes qui suivent `'Python'`.
2. Écrire une requête qui donne toutes les personnes qui aiment `'IA'`.
3. Écrire une requête qui donne les personnes qui suivent `'Python'` **et** aiment `'IA'`.
4. Quels sont les résultats attendus ?

4. Écriture de règles

On reprend les faits suivants :

```
from pyDatalog import pyDatalog
pyDatalog.clear()
pyDatalog.create_terms('X, suit, aime, interesse_par_ia')

+suit('Alice', 'Python')
+suit('Bob', 'Python')
+suit('Claire', 'Logique')
+aime('Alice', 'IA')
+aime('Bob', 'Reseaux')
+aime('Claire', 'IA')
```

On veut exprimer :

— une personne est intéressée par l'IA si elle aime l'IA ;
— une personne est aussi intéressée par l'IA si elle suit un cours de logique.

1. Écrire la première règle.
2. Écrire la deuxième règle.
3. Donner une requête permettant d'obtenir toutes les personnes intéressées par l'IA.
4. Quels sont les résultats attendus ?

5. Fonctions logiques

On considère :

```
from pyDatalog import pyDatalog
pyDatalog.clear()
pyDatalog.create_terms('X, Y, note, bonus, note_finale')
```

```
note['Alice'] = 12
note['Bob'] = 9
note['Claire'] = 15
bonus[None] = 1
```

1. Écrire une requête qui donne la note de `'Alice'`.
2. Définir une fonction logique `note_finale[X]` qui vaut `note[X] + bonus[None]`.
3. Donner une requête qui affiche les notes finales de tous les étudiants.
4. Donner une requête qui cherche les étudiants dont la note finale est supérieure ou égale à 10.

6. Agrégation simple

On considère :

```
from pyDatalog import pyDatalog
pyDatalog.clear()
pyDatalog.create_terms('X, Y, suit, nb_etudiants, len_')

+suit('Alice', 'Python')
+suit('Bob', 'Python')
+suit('Claire', 'Logique')
+suit('David', 'Python')
+suit('Emma', 'Logique')
```

1. Définir une agrégation `nb_etudiants[Y]` qui compte le nombre d'étudiants inscrits dans chaque cours.
2. Quelle valeur obtient-on pour `'Python'` ?
3. Quelle valeur obtient-on pour `'Logique'` ?

7. Compléter un petit programme

Compléter le code suivant pour exprimer :

— Alice lit ;
— Bob lit ;
— si quelqu'un lit, alors cette personne apprend.

```
from pyDatalog import pyDatalog
pyDatalog.clear()
pyDatalog.create_terms('X, lit, apprend')

# faits
________________________
________________________

# regle
________________________

# requete
print(___________________)
```

1. Compléter le programme.
2. Que renvoie la dernière ligne ?

Corrigé

1. Questions de cours — Correction

1. Un **fait** est une information déclarée comme vraie dans la base. Une **règle** permet de déduire une nouvelle information à partir d'autres informations.
2. `pyDatalog.create_terms(...)` sert à déclarer les termes que l'on va utiliser : variables, prédicats, fonctions logiques, etc.
3. Les variables logiques commencent généralement par une majuscule par convention, et parce que cela permet de les distinguer des noms de prédicats ou de fonctions.
4. Le symbole `<=` se lit ici comme « est vrai si » et sert à écrire une règle.

2. Premières requêtes — Correction

1.
```
print(X == 3)
```
2.
```
print((X == 2) & (Y == X + 1))
```
3. On attend :

$$X = 2 \qquad \text{et} \qquad Y = 3$$

3. Faits et requêtes — Correction

1.
```
print(suit(X, 'Python'))
```
2.
```
print(aime(X, 'IA'))
```
3.
```
print(suit(X, 'Python') & aime(X, 'IA'))
```
4. Résultats attendus :
   - pour `suit(X, 'Python')` : `Alice`, `Bob` ;
   - pour `aime(X, 'IA')` : `Alice`, `Claire` ;
   - pour la conjonction des deux : `Alice`.

4. Écriture de règles — Correction

1. Première règle :
```
interesse_par_ia(X) <= aime(X, 'IA')
```
2. Deuxième règle :
```
interesse_par_ia(X) <= suit(X, 'Logique')
```
3. Requête :
```
print(interesse_par_ia(X))
```
4. Résultats attendus :
   - `Alice`, car elle aime l'IA ;
   - `Claire`, car elle aime l'IA et suit aussi `Logique`.

5. Fonctions logiques — Correction

1. Requête pour la note d'Alice :

```
print(note['Alice'] == X)
```

2. Définition :

```
note_finale[X] = note[X] + bonus[None]
```

3. Requête :

```
print(note_finale[X] == Y)
```

4. Requête :

```
print((note_finale[X] == Y) & (Y >= 10))
```

On obtient les notes finales suivantes :

— Alice : 13
— Bob : 10
— Claire : 16

Les étudiants dont la note finale est supérieure ou égale à 10 sont donc : `Alice`, `Bob` et `Claire`.

6. Agrégation simple — Correction

1. Définition :

```
(nb_etudiants[Y] == len_(X)) <= suit(X, Y)
```

2. Pour `'Python'`, on obtient : 3

```
print(nb_etudiants['Python'] == X)
```

car Alice, Bob et David suivent ce cours.

3. Pour `'Logique'`, on obtient : 2

```
print(nb_etudiants['Logique'] == X)
```

car Claire et Emma suivent ce cours.

7. Compléter un petit programme — Correction

1. Une solution possible :

```
from pyDatalog import pyDatalog
pyDatalog.clear()
pyDatalog.create_terms('X, lit, apprend')

# faits
+lit('Alice')
+lit('Bob')
```

```
# regle
apprend(X) <= lit(X)

# requete
print(apprend(X))
```

2. La dernière ligne renvoie les personnes pour lesquelles on peut déduire `apprend(X)`, donc ici : `Alice` et `Bob`.